COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/17/26	**DATE SUBMITTED:**	07/22/26
NAME:	Villaruel, Prince Benjamin S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 2-1. Initial Driver Program

Screenshot

main.cpp

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > HOA 2.1 > main.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
public:
    //costructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };
    //deconstructor
    ~Student(){
        std::cout << "Deconstructor Called." << std::endl;
    }
    //copy constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //copy assignment operator
    //called when an object already existsis overwritten
    Student& operator=(const Student &copyStudent){
        std::cout << "Copy Assignment Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
        return *this;
    }
    //display attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    //called the copy assingment operator that I created since s3 already exists
    student3 = student2;

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 2.1>
Constructor Called.
Copy Constructor Called.
Constructor Called.
Copy Assignment Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
```

Observation	The instructions only defined the constructor, destructor, and copy constructor, so I added the copy assignment operator myself to properly observe when student 3 = student 2 runs.

Table 2-2. Modified Driver Program with Student Lists

Screenshot

main.cpp

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > HOA 2.1 > main.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
public:
    //costructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };
    //deconstructor
    ~Student(){
        std::cout << "Deconstructor Called." << std::endl;
    }
    //copy constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //copy assignment operator
    //called when an object already existsis overwritten
    Student& operator=(const Student &copyStudent){
        std::cout << "Copy Assignment Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
        return *this;
    }
    //display attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zach", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 2.1> cd "c:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 2.1\" ;
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
```

Observation	After I modified the main function of the first part, Student studentList[j] = {}; immediately called the constructor five times using default values before any names or ages were set. All

	five Student objects were then destroyed together when the main function ended.

Table 2-3. Final Driver Program

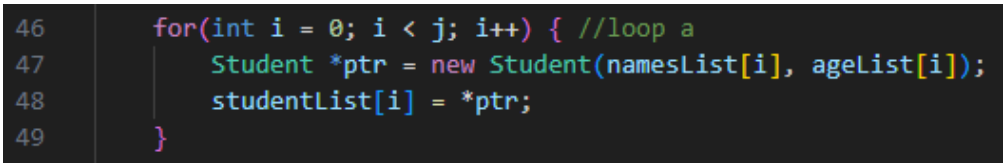
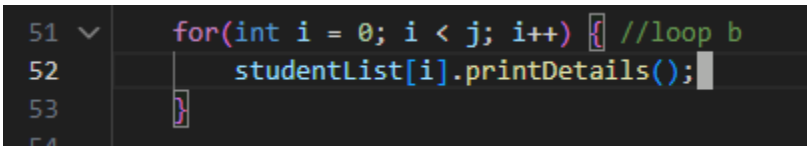
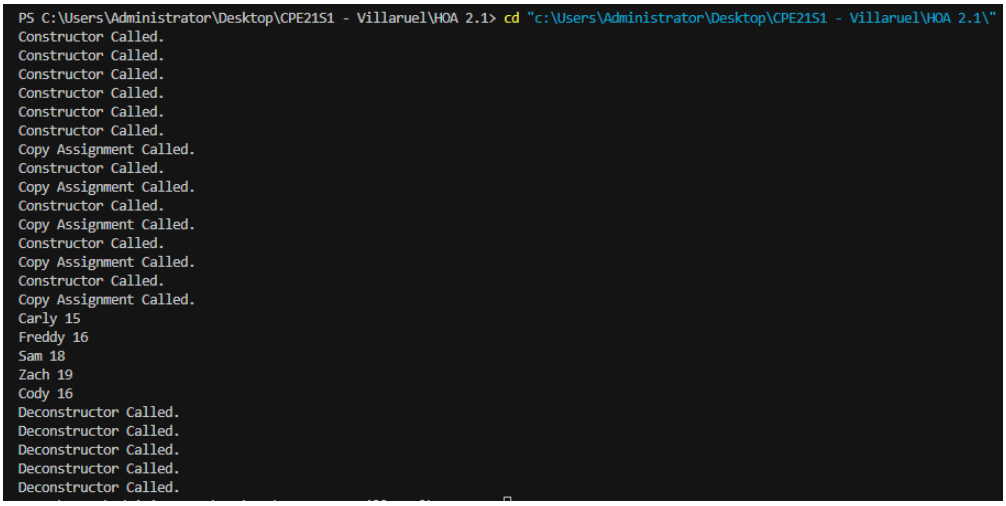
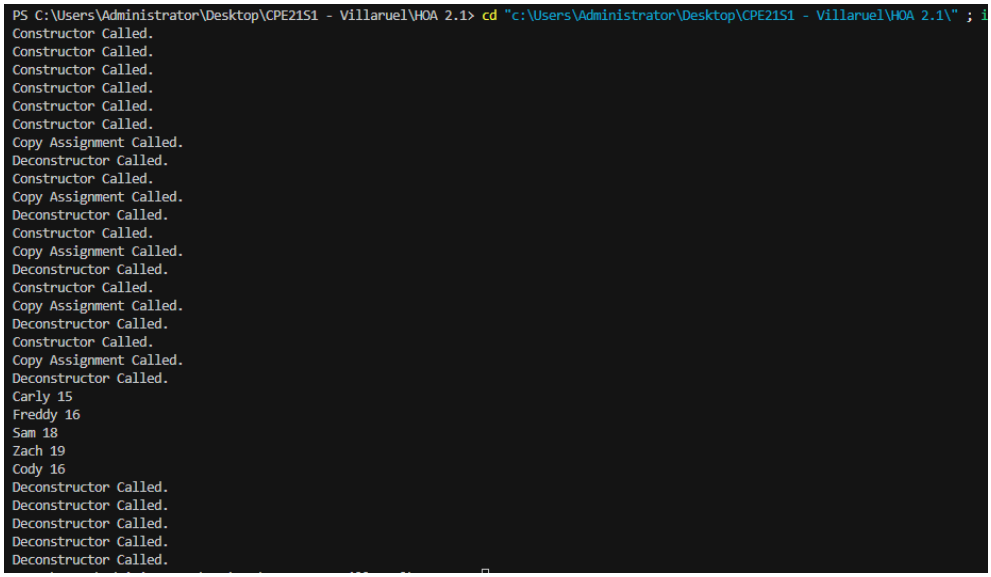
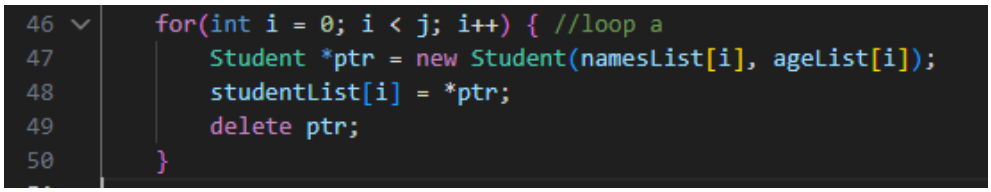
Loop A	
Observation	It creates a new student on the heap, then copies its data into studentList[i], which already exists and it declares copy assignment instead of copy constructor.
Loop B	
Observation	It prints each student's name and age using printDetails().
Output	
Observation	Despite the names and ages are displayed successfully, the heap objects from new are never deleted, causing a memory leak.

Table 2-4. Modifications/Corrections Necessary

Modifications	 
Observation	We have now complete destructor calls, 5 for the temporary heap objects freed inside loop a, and 5 for studentList array elements at program end.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above

supp.cpp

```cpp
#include <iostream>
#include <string.h>

//PROBLEM 1
class Fruits {
private:
    std::string name;
    int price, quantity;
public:
//constuctor
    Fruits(std::string newName = "Unknown", int newPrice = 0, int newQuantity = 0){
        name = newName;
        price = newPrice;
        quantity = newQuantity;
        std::cout << "Fruits Constructor Called." << std::endl;
    }
//destructor
    ~Fruits() {std::cout << "Fruits Destructor Called." << std::endl;}
//copy constructor
    Fruits(const Fruits &copyFruits){
        std::cout << "Fruits Copy Constructor Called." << std::endl;
        name = copyFruits.name;
        price = copyFruits.price;
        quantity = copyFruits.quantity;
    }
//copy assignment operator
    Fruits& operator = (const Fruits &copyFruits){
        std::cout << "Fruits Copy Assignmment Operator Called." << std::endl;
        name = copyFruits.name;
        price = copyFruits.price;
        quantity = copyFruits.quantity;
        return *this;
    }
//calculate sum
    int calculateSum() {return price * quantity;}
    //display attributes
    void printDetails(){
        std::cout << name << " - PHP " << price << " x" << quantity
                  <<" = PHP " << calculateSum() << std::endl;
    }
};

class Vegetables {
private:
    std::string name;
    int price, quantity;
public:
//constuctor
    Vegetables(std::string newName = "Unknown", int newPrice = 0, int newQuantity = 0){
        name = newName;
        price = newPrice;
        quantity = newQuantity;
        std::cout << "Vegetables Constructor Called." << std::endl;
    }
//destructor
    ~Vegetables() {std::cout << "Vegetables Destructor Called." << std::endl;}
//copy constructor
    Vegetables(const Vegetables &copyVegetables){
        std::cout << "Vegetables Copy Constructor Called." << std::endl;
        name = copyVegetables.name;
        price = copyVegetables.price;
        quantity = copyVegetables.quantity;
    }
//copy assignment operator
    Vegetables& operator = (const Vegetables &copyVegetables){
        std::cout << "Vegetables Copy Assignmment Operator Called." << std::endl;
        name = copyVegetables.name;
        price = copyVegetables.price;
        quantity = copyVegetables.quantity;
        return *this;
    }
//calculate sum
    int calculateSum() {return price * quantity;}
    //display attributes
    void printDetails(){
        std::cout << name << " - PHP " << price << " x" << quantity <<" = PHP " << calculateSum() << std::endl;
    }
};
```

Observation: For problem 1, I created separate Fruits and Vegetables classes, each with a constructor, deconstructor, copy constructor, copy assignment operator, following the same procedure that I did earlier.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
92  //PROBLEM 2
93  int main() {
94      const int fruitsCount = 2;
95      const int vegetablesCount = 2;
96
97      Fruits* fruitsList[fruitsCount];
98      fruitsList[0] = new Fruits("Apple", 10, 7);
99      fruitsList[1] = new Fruits("Banana", 10, 8);
100
101     Vegetables* vegetablesList[vegetablesCount];
102     vegetablesList[0] = new Vegetables("Lettuce", 50, 10);
103     vegetablesList[1] = new Vegetables("Broccoli", 60, 12);
104 //access each saved and display all details
105     std::cout << "\n Jenna's Grocery List " << std::endl;
106     for (int i = 0; i < fruitsCount; i++) fruitsList[i]->printDetails();
107     for (int i = 0; i < vegetablesCount; i++) vegetablesList[i]->printDetails();
108 //display total sum
109     std::cout << "\nTotal (w/ Lettuce): PHP " << totalSum(fruitsList, fruitsCount, vegetablesList, vegetablesCount) << std::endl;
110
```

Observation: For problem 2, I built fruitsList and vegetablesList as arrays of pointers, dynamically allocating each item with new so individual items can later be deleted separately in problem 4. Looping through each array and calling printDetails() correctly displayed all four items with their name, price, quantity and total.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

```
80  //PROBLEM 3
81  int totalSum(Fruits* fruitsList[], int fruitsCount, Vegetables* vegetablesList[], int vegetablesCount) {
82      int total = 0;
83      for (int i = 0; i < fruitsCount; i++) {
84          if (fruitsList[i] != nullptr) total += fruitsList[i]->calculateSum();
85      }
86      for (int i = 0; i < vegetablesCount; i++) {
87          if(vegetablesList[i] != nullptr) total += vegetablesList[i]->calculateSum();
88      }
89      return total;
90  }
91
```

Observation: For problem 3, I wrote totalSum as a stand alone function that takes both pointer arrays and their sizes as parameters, looping through each and calling calculateSum() (defined in problem 1) on every item to accumulate the total.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```cpp
//PROBLEM 4
    std::cout << "\nRemoving Lettuce " << std::endl;
    delete vegetablesList[1]; //free lettuce memory usage
    vegetablesList[1] = nullptr; //clear the pointer

    std::cout << "\nTotal (w/o Lettuce): PHP " << totalSum(fruitsList, fruitsCount, vegetablesList, vegetablesCount) << std::endl;

//clean up
    for (int i = 0; i < fruitsCount; i++) delete fruitsList[i];
    for (int i = 0; i < vegetablesCount; i++) {
       if (vegetablesList[i] != nullptr) delete vegetablesList[i];
    }

    return 0;

}
```

Observation: For problem 4, I used delete vegetablesList, to remove lettuce dynamically allocated memory, then set the pointer to nullptr to avoid leaving a random pointer that could be accidentally accessed later.

Output

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 2.1>
Fruits Constructor Called.
Fruits Constructor Called.
Vegetables Constructor Called.
Vegetables Constructor Called.

 Jenna's Grocery List
Apple - PHP 10 x7 = PHP 70
Banana - PHP 10 x8 = PHP 80
Lettuce - PHP 50 x10 = PHP 500
Broccoli - PHP 60 x12 = PHP 720

Total (w/ Lettuce): PHP 1370

Removing Lettuce
Vegetables Destructor Called.

Total (w/o Lettuce): PHP 650
Fruits Destructor Called.
Fruits Destructor Called.
Vegetables Destructor Called.
```

Conclusion: Throughout this laboratory, I learned how the constructor, destructor, copy constructor, and copy assignment operator each function automatically at different moments in an object's life, and I managed to confirm this by tracing cout output against the driver code. The procedure moved from a simple Student class using arrays to allocated objects with new and delete, teaching me the need of proper de-allocation after I delete ptr was missing, and had to fix it. The supplementary activity reinforced these concepts in a more practical way, applying them to two separate classes for the grocery list, especially through writing totalSum and properly deleting a single item without breaking the rest. Overall, I think I did fairly well, though small mistakes like a mistyped loop condition and a missing member function after duplicating a class showed me I need to double check everything before compiling.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **grammar correction, explanation of concepts, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **There's some concepts and errors that I wasn't able to fix and understand by myself and required support from different tools, in order for me to progress and learn new things and apply them in future encounters.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.